

Representação Matricial de Imagens Digitais

Entrega 1 — Álgebra Linear, Vetores e Geometria Analítica

Integrante	RA
Felipe Vallim Soares	24026060
Guilhermy Mariano Lisboa Garcia	23025371
Gustavo Oliveira Demetrio	24026213
Saulo Pereira de Jesus	24026095

1. Introdução

No contexto da disciplina de Álgebra Linear, Vetores e Geometria Analítica, este documento apresenta o primeiro entregável do projeto aplicado do semestre. O objetivo é demonstrar que toda imagem digital pode ser representada como uma matriz numérica — cada pixel corresponde a um vetor (R, G, B) com valores inteiros entre 0 e 255, e uma imagem $H \times W$ equivale a uma matriz $H \times W$ de vetores tridimensionais. Para comprovar isso, desenvolvemos um pipeline em Python que carrega uma imagem do projeto do semestre (Lideranças Empáticas — Contagem Inteligente de Alimentos), extrai os pixels para CSV e reconstrói a imagem a partir dos números, sem perda de informação.

2. Objetivo

Introduzir a interpretação de imagens digitais como estruturas matriciais, estabelecendo a base conceitual para a aplicação de Álgebra Linear em contextos computacionais. Ao final deste entregável, espera-se compreender que imagens digitais podem ser formalmente tratadas como matrizes e vetores, reconhecendo a Álgebra Linear como a linguagem matemática subjacente à representação de dados visuais em sistemas computacionais.

3. Divisão de Responsabilidades

Felipe Vallim — Responsável pelo desenvolvimento do módulo de carregamento e redimensionamento da imagem. Implementou a lógica de busca automática de imagens no diretório `images/`, a conversão para o modo RGB e o redimensionamento padronizado para 50×50 pixels utilizando a biblioteca Pillow. Também contribuiu com a seleção da imagem de entrada relacionada ao projeto do semestre.

Guilhermy Mariano — Responsável pela extração da matriz de pixels e pela exportação dos dados para CSV. Desenvolveu a lógica de percorrimento pixel a pixel da imagem, montando o DataFrame com as colunas Y, X, R, G e B utilizando a biblioteca pandas, e implementou a função de salvamento do DataFrame em arquivo CSV.

Gustavo Oliveira — Responsável pelo módulo de reconstrução da imagem a partir do CSV. Implementou a leitura do arquivo CSV, a criação do array NumPy com as dimensões corretas e o preenchimento das posições (Y, X) com os valores RGB, gerando a imagem reconstruída via `Image.fromarray`. Também realizou a verificação numérica de igualdade entre a imagem original redimensionada e a reconstruída utilizando `np.array_equal`.

Saulo Pereira — Responsável pela orquestração geral do pipeline (`main.py`), pela elaboração do Jupyter Notebook interativo com todas as visualizações e comparações documentadas passo a passo, e pela redação e formatação do presente documento. Também desenvolveu o script de geração da planilha Excel, que produz o arquivo `representacao_matricial.xlsx` com duas abas: "Dados dos Pixels" e "Imagem Reconstruída".

4. Metodologia e Implementação

A solução foi desenvolvida em Python, utilizando uma arquitetura modular com responsabilidades bem definidas. Cada etapa do processo é encapsulada em um módulo específico, garantindo clareza, reusabilidade e facilidade de manutenção. Além dos módulos Python, foi desenvolvido um Jupyter Notebook interativo que reproduz todo o processo com visualizações e explicações detalhadas. A seguir, descrevemos cada etapa do pipeline.

4.1 Carregamento e Redimensionamento da Imagem

O módulo `image_loader.py` é responsável por localizar automaticamente a primeira imagem disponível no diretório `images/` (suportando os formatos PNG, JPG, JPEG e BMP) e carregá-la utilizando a biblioteca Pillow (PIL). A imagem é convertida para o modo RGB, garantindo uniformidade independente do formato original, e redimensionada para 50×50 pixels.

O redimensionamento para 50×50 é intencional: gera uma matriz de exatamente 2.500 pixels (2.500 linhas no CSV), um tamanho controlado que facilita a visualização e análise dos dados sem comprometer a demonstração do conceito.

4.2 Extração da Matriz de Pixels

O módulo `pixel_extractor.py` percorre cada pixel da imagem, linha por linha (Y) e coluna por coluna (X), extraindo seus valores de cor. Cada pixel se torna um registro com cinco campos:

Y (posição vertical / linha da matriz), **X** (posição horizontal / coluna da matriz), **R** (componente vermelha, 0–255), **G** (componente verde, 0–255) e **B** (componente azul, 0–255). Os dados são organizados em um DataFrame do pandas.

Essa extração demonstra que a imagem é, literalmente, uma tabela de números organizados em formato matricial: cada posição (Y, X) da matriz contém um vetor (R, G, B).

4.3 Exportação para CSV

O módulo `data_exporter.py` salva o DataFrame como um arquivo CSV (`pixel_data.csv`). Este arquivo contém toda a informação numérica da imagem e pode ser aberto em qualquer editor de planilhas (Excel, Google Sheets, LibreOffice) para verificação manual.

O CSV evidencia a correspondência direta entre a imagem original e sua representação matricial: cada linha do arquivo corresponde a um pixel, e cada pixel é descrito por sua posição na matriz (Y, X) e seu vetor de cor (R, G, B). A imagem inteira se reduz a uma tabela de números.

4.4 Reconstrução da Imagem a Partir do CSV

O módulo `image_reconstructor.py` realiza o processo inverso: lê exclusivamente o arquivo CSV (ignorando completamente a imagem original), determina as dimensões da imagem a partir dos valores máximos de Y e X, cria um array NumPy de zeros com essas dimensões e preenche cada posição com os valores RGB correspondentes. O resultado é uma imagem reconstruída pixel por pixel.

A verificação numérica (realizada no Jupyter Notebook) confirma que a imagem original redimensionada e a imagem reconstruída são numericamente idênticas — provando que a matriz numérica contém 100% da informação da imagem, sem qualquer perda de dados.

4.5 Geração da Planilha Excel

O módulo `generate_excel.py` lê o `pixel_data.csv` e gera a planilha "representacao_matricial.xlsx" com a biblioteca `openpyxl`. O arquivo possui duas abas: "Dados dos Pixels", com as 2.500 linhas contendo Y, X, R, G e B, e "Imagem Reconstruída", uma grade 50×50 onde cada célula é pintada com a cor RGB do pixel correspondente, formando a imagem diretamente na planilha.

5. Estrutura do Código

O projeto está organizado nos seguintes arquivos:

Arquivo	Responsabilidade
<code>main.py</code>	Orquestra todo o pipeline: localiza a imagem, redimensiona, extrai pixels, exporta CSV e reconstrói a imagem
<code>image_loader.py</code>	Busca a primeira imagem no diretório e a carrega/redimensiona para 50×50 pixels
<code>pixel_extractor.py</code>	Percorre a imagem pixel a pixel e monta o DataFrame com coordenadas (Y, X) e cores (R, G, B)
<code>data_exporter.py</code>	Exporta o DataFrame para arquivo CSV
<code>image_reconstructor.py</code>	Lê o CSV e reconstrói a imagem usando NumPy e Pillow
<code>rgb_pixel_extraction.ipynb</code>	Notebook interativo com todo o processo documentado passo a passo, incluindo visualizações e comparações
<code>generate_excel.py</code>	Lê o CSV e gera o arquivo <code>representacao_matricial.xlsx</code> com duas abas: tabela de dados dos pixels e grade 50×50 com células coloridas pelo valor RGB correspondente

6. Resultados Visuais

Nesta seção são apresentados os resultados visuais obtidos durante a execução do pipeline. As imagens demonstram cada etapa do processo, desde a imagem original até sua reconstrução a partir dos dados numéricos.

6.1 Imagem Original (Redimensionada)

A imagem abaixo corresponde à imagem selecionada para o projeto, após o carregamento e redimensionamento para 50×50 pixels. Esta é a imagem de entrada do pipeline, a partir da qual os pixels foram extraídos.



Figura 1 — Imagem original redimensionada para 50×50 pixels

6.2 Trecho do Arquivo CSV (Representação Matricial)

O trecho abaixo ilustra as primeiras linhas do arquivo CSV gerado, evidenciando a estrutura tabular com as coordenadas (Y, X) e os valores de cor (R, G, B) de cada pixel. Cada linha do CSV corresponde a um elemento da matriz, e o arquivo completo contém 2.500 linhas (50 linhas × 50 colunas).

	Y	X	R	G	B
0	0	0	255	255	255
1	0	1	255	255	255
2	0	2	255	255	255
3	0	3	255	255	255
4	0	4	255	255	255
5	0	5	255	255	255
6	0	6	255	255	255
7	0	7	255	255	255
8	0	8	255	255	255
9	0	9	255	255	255

Figura 2 — Trecho do arquivo `pixel_data.csv` evidenciando a representação matricial

6.3 Imagem Reconstruída a Partir do CSV

A imagem abaixo foi reconstruída exclusivamente a partir dos dados numéricos do arquivo CSV, sem qualquer acesso à imagem original. O fato de ser visualmente idêntica à original comprova que toda a informação visual estava integralmente codificada na estrutura matricial.

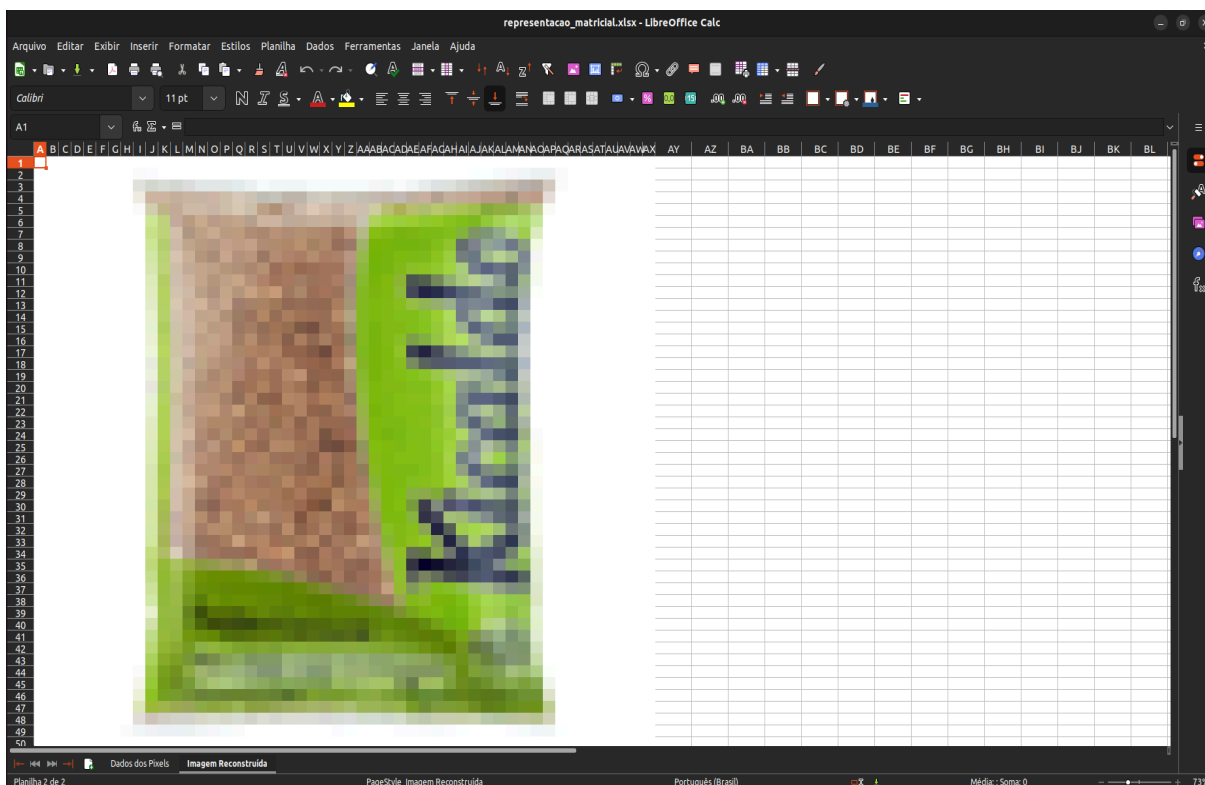


Figura 3 — Imagem reconstruída exclusivamente a partir do CSV

6.4 Comparação: Original vs. Reconstruída

A figura abaixo apresenta a comparação lado a lado entre a imagem original (redimensionada) e a imagem reconstruída a partir do CSV. A verificação numérica realizada no notebook confirma que ambas são numericamente idênticas (todos os 2.500 pixels coincidem em seus valores R, G e B).



Figura 4 — Comparação lado a lado: original (esquerda) vs. reconstruída do CSV (direita)

7. Conclusão

Este trabalho demonstrou, de forma prática e computacional, que uma imagem digital de dimensão $H \times W$ é equivalente a uma matriz $H \times W$, onde cada elemento é um vetor de 3 componentes (R, G, B) com valores inteiros no intervalo $[0, 255]$.

A conversão completa da imagem para uma tabela numérica (CSV) e sua subsequente reconstrução perfeita comprovaram que toda a informação visual está integralmente codificada na estrutura matricial, sem qualquer perda de dados. A imagem original e a reconstruída foram verificadas como numericamente idênticas.

Portanto, concluímos que uma imagem digital é, por definição, uma matriz de números, e a Álgebra Linear é a linguagem matemática subjacente à representação de dados visuais em sistemas computacionais.